Practical LAMBDA Programming

Building Real Systems in Excel & Google Sheets

Part 2 of 2 · Applications & Practice

Leonard Brown · April 2026 [3sigmalectures@gmail.com]

FOUNDATION

A LAMBDA Is Just a Function Definition

Mathematical Form

```
f(x) = x + 1
f(2) = 3
```

Spreadsheet Form

```
=LAMBDA(x, x+1)
=LAMBDA(x, x+1)(2)  →  3
```

General Form

```
=LAMBDA( param1, param2, ... , function_expression )

f(param1, param2, ...) = function_expression (param1, parma2, ... )
```

KEY IDEA

A LAMBDA is just a function definition — parameters plus a function expression.

FOUNDATION

Functions with Multiple Parameters

Mathematical Form

```
f(x, y) = x + y          f(2, 3) = 5
g(x, y, z) = x + y + z   g(1, 2, 3) = 6
```

Spreadsheet Form

```
=LAMBDA(x, y, x + y)(2, 3)        →  5
=LAMBDA(x, y, z, x + y + z)(1, 2, 3) →  6
```

Calling a LAMBDA

```
=LAMBDA(x, x+1)(5)  →  6       // Define and execute immediately — no name required
```

KEY IDEA

A LAMBDA consists of parameters + a function expression, just like a regular function definition.

FOUNDATION

Named Functions

Excel

Name Manager
(Formulas → Define Name)

Google Sheets

Named Functions
(Data → Named Functions)

- Define once
- Reuse anywhere in the workbook
- Share across sheets

STRUCTURE

Introducing LET

LET returns a function expression. It allows you to break a LAMBDA's function expression into named sub-expressions.

Spreadsheet

```
=LET(
  x,      5,
  result, x + 5,
  result
)
```

Equivalent Code

```
{
  x      = 5;
  result = x + 5;
  result;
}
```

KEY IDEA

LET lets you break and expression, into sub-expressions, and name the sub-expressions. The value of the LET is the last sub-expresion. IN standard-code, Let is equivalent to a code block delimited by curly braces.

BEST PRACTICE

The "result" Pattern

Always make the last sub-expression in a LET, be a single variable name (example “result”).

```
=LET(
  x,      5,
  y,      10,
  result, x + y,
  result          // ← always return result
)
```

Why this matters

- Single inspection point
- Easy to isolate errors
- Step-by-step validation
- Prevents nested formula complexity
- Acts like a return variable

ARRAY FUNCTIONS

MAP — Apply a Function to Every Element

```
=MAP(A1:A10,
  LAMBDA(x,
    LET(
      result, x * 2,
      result
    )
  )
)
```

Input

Accepts an array of values

Apply

Applies a LAMBDA to each element

Output

Returns an array of the same size

KEY IDEA

MAP transforms every element of an array without loops.

ARRAY FUNCTIONS

REDUCE — Aggregate an Array to a Single Value

```
=REDUCE(0, A1:A10,
  LAMBDA(acc, x,
    LET(
      result, acc + x,
      result
    )
  )
)
```

Initial Value

Starts with an accumulator (here: 0)

Iterate

Applies LAMBDA sequentially to each element

Accumulate

Updates acc at each step

Output

Returns a single final accumulated value

KEY IDEA

REDUCE combines an array into a single result through iterative accumulation.

ARRAY FUNCTIONS

SCAN — Running Accumulation

```
=SCAN(0, A1:A10,
  LAMBDA(acc, x,
    LET(
      result, acc + x,
      result
    )
  )
)
```

SCAN vs REDUCE

	REDUCE	SCAN
Input	array	array
Output	1 value	array (same size)
Shows	final acc	all intermediate values
Use for	totals	running totals / debugging

KEY IDEA

SCAN is like REDUCE, but returns all intermediate values — ideal for running totals and debugging.

ADVANCED

Higher-Order Functions

```
=MAP(range, LAMBDA(x, f(x)))
```

Functions are first-class objects:

Passed as inputs

A LAMBDA can accept another LAMBDA as a parameter

Returned as outputs

A LAMBDA can return another LAMBDA

Composed

Functions can be chained to build complex behavior

KEY IDEA

This is real programming — LAMBDA enables functional composition inside a spreadsheet.

ADVANCED

Recursion

A recursive function calls itself. Example: sumn(x) — sums integers from n down to 1.

```
=LAMBDA(x,
  IF(
    x = 1,
    1,                  // base case
    x + sumn(x - 1)     // recursive step
  )
)
```

How it works

- Base case: x = 1 → return 1
- Recursive step: return x + sumn(x−1)
- Reduces the problem at each step
- Must be saved as a named function to self-reference

KEY IDEA

Solve a problem by reducing it to a smaller version of itself.

ARCHITECTURE

The Function Library Pattern

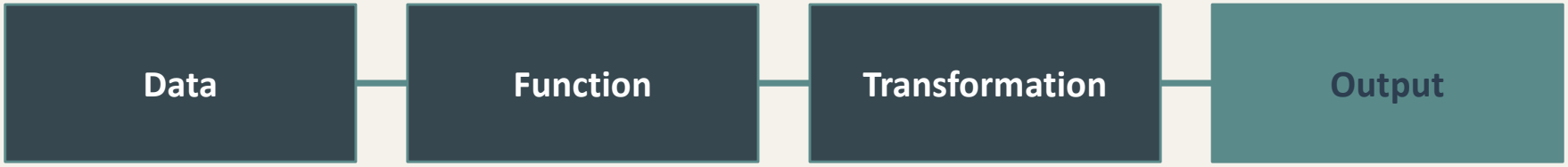
Managing LAMBDAs scattered across Name Manager is hard. The solution: a centralised FUNCTION_LIB.

Library = source of truth · Workbook = execution environment

- Centralised control
- Transparent design
- Version control
- Reusable across workbooks
- Easier debugging
- Portability

CONCEPT

The Functional Dataflow Model



- No explicit loops — iteration is expressed through array functions
- No mutation — values are transformed, not overwritten
- Computation expressed as pure transformations of data
- Programs are built by composing functions

Key Takeaway

LAMBDA turns the spreadsheet into a functional programming environment.

- Functions are the primary building blocks
- Iteration expressed without loops
- Complex formulas decomposed into small named sub-expressions
- Define once, reuse everywhere
- LAMBDA is a complete programming model — any computable function can be expressed using lambda calculus

Spreadsheets are evolving into programmable environments.

LAMBDA provides:

Structure

Abstraction

Scalability

Leonard Brown · April 2026